# YMÜ227 NESNE TABANLI PROGRAMLAMA

## PROJE RAPORU

### Proje Adı: Metin Tabanlı Mini–Macera Oyunu (Java)

| | |
|---|---|
| **Ders Adı:** | YMÜ227 Nesne Tabanlı Programlama |
| **Öğretim Elemanı:** | Sinem AKYOL / Nigar ÖZBEY |
| **Öğrenci Adı Soyadı:** | Abdullah Batur DEMİR |
| **Öğrenci Numarası:** | 240290040 |
| **Teslim Tarihi:** | 30.11.2025 |

# İçindekiler


1. Giriş
2. Proje Amacı
3. Geliştirme Süreci
    a. Uygulanan OOP Prensipleri
    b. Sınıf Tasarımı
4. Kaynak Kodlar Hakkında
5. Çıktılar (Oyun Senaryoları)
6. Sonuç ve Değerlendirme


---

# 1. Giriş

Bu proje, YMÜ227 Nesne Tabanlı Programlama dersi kapsamında, Java programlama dili kullanılarak geliştirilmiş konsol tabanlı bir Mini-Macera Oyunu'dur. Oyun, oyuncunun birbirine bağlı odalardan oluşan sanal bir dünyada komutlar aracılığıyla (örneğin: `go kuzey`, `take key`, `talk muhafiz`) etkileşim kurmasını sağlar. Projenin temel amacı, nesne tabanlı programlama (OOP) kavramları olan Kalıtım (Inheritance), Çok Biçimlilik (Polymorphism) ve Kompozisyon (Composition) ilkelerini pratik bir uygulama üzerinden göstermektir.

Oyun, manuel ID ataması yapılmış odalardan oluşan bir harita, farklı eşya türleri (`Item` alt sınıfları), karakter türleri (`Npc1` alt sınıfları) ve özel mekanikler (Tuzak Odası gibi) içerir. Tüm işlemler `GameEngine` sınıfı tarafından merkezi olarak yönetilmektedir.

# 2. Proje Amacı

Proje, aşağıdaki teknik ve öğrenim amaçlarını gerçekleştirmek için tasarlanmıştır:

- **Kalıtım (Inheritance):** `Item`, `Room` ve `Npc1` gibi temel sınıflardan özelleşmiş alt sınıflar türetmek.
- **Çok Biçimlilik (Polymorphism):** Nesnelerin çalışma zamanında türlerine göre farklı davranışlar sergilemesini sağlamak (Örn: `onUse()` metodu).
- **Kompozisyon (Composition):** Sınıfların diğer nesneleri (Liste, Harita vb.) barındırarak karmaşık yapılar oluşturması.
- **Hata Yönetimi:** Kullanıcı hatalarını ve sistemsel istisnaları `try-catch` blokları ile yönetmek.
- **Veri Yapıları:** `List`, `Map` gibi koleksiyonların etkin kullanımı.

# 3. Geliştirme Süreci

## 3.1. Uygulanan Nesne Tabanlı Programlama Prensipleri

| Prensip | Uygulama Detayı |
|---|---|
| **Kalıtım** | `Item` (Soyut) -> `KeyItem`, `PotionItem`, `WeaponItem`<br>`NPC` (Soyut) -> `FriendlyNPC`, `EnemyNPC`<br>`Room` -> `TrapRoom` |
| **Çok Biçimlilik** | `Item` sınıfındaki soyut `onUse()` metodu her alt sınıfta ezilmiştir (Override). `KeyItem` kilit açarken, `PotionItem` can verir. Benzer şekilde `TrapRoom` sınıfı, `Room` sınıfından farklı davranışlar sergiler. |
| **Kompozisyon** | `Room` sınıfı, içinde `Item` ve `NPC` listeleri barındırır. `GameEngine` sınıfı tüm oyun dünyasını (Map) yönetir. |

## 3.2. Sınıf Tasarımı ve Görevleri

| Sınıf Adı | Görevi |
|---|---|
| `Room` | Oda bilgilerini, çıkışları ve içindeki nesneleri tutar. |
| `TrapRoom` | `Room` sınıfından türetilmiştir. Odaya girildiğinde oyunu sonlandıran tuzak mekaniğini içerir. |
| `Item` (Abstract) | Tüm eşyaların atasıdır. ID, isim ve açıklama tutar. |
| `KeyItem` | Belirli bir kapıyı açmak için kullanılır. |
| `PotionItem` | Oyuncunun can puanını (HP) artırır. |
| `WeaponItem` | Oyuncunun saldırı gücünü artırır. |

| | |
|---|---|
| `Npc1` (Abstract) | Karakterlerin temel sınıfıdır. |
| `FriendlyNPC` | Diyalog ağacı yapısını kullanarak oyuncuyla konuşur. |
| `EnemyNPC` | Oyuncuya saldırır ve hasar verir. |
| `GameEngine` | Oyunun beynidir. Komutları işler, haritayı oluşturur ve tekil `Scanner` yapısı ile girdileri yönetir. |

## 4. Kaynak Kodlar

Projenin kaynak kodları (`.java` dosyaları) bu raporun ekinde sunulan ZIP dosyası içerisinde yer almaktadır. Kodlar, Java isimlendirme standartlarına uygun yazılmış ve gerekli yorum satırları eklenmiştir.

# Room.java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;


/*
 * Oyun dünyasındaki bir odayı temsil eden kodlar.
 * Oda içeriğini(NPC,eşyalar) tutar.
 */
public class Room {
    private final int id;
    private final String name;
    private final String description;
    // Çıkışlar (kuzey, guney, ...)
    private final Map<String, Room> exits;
    // Kilitli Çıkışlar ("kirmizi_kapi")
    private final Map<String, String> lockedExits;
    private final List<Item> items;
    private final List<Npc1> npcs;

    public Room(int id, String name, String description) {
        this.id = id;
        this.name = name;
        this.description = description;
```

```java
        this.exits = new HashMap<>();
        this.lockedExits = new HashMap<>();
        this.items = new ArrayList<>();
        this.npcs = new ArrayList<>();
    }
    // Dışarıdan erişim için yazılan metotlar.
    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Map<String, Room> getExits() {
        return exits;
    }

    public Map<String, String> getLockedExits() {
        return lockedExits;
    }

    public List<Item> getItems() {
        return items;
    }

    public List<Npc1> getNpcs() {
        return npcs;
    }

    // İki oda arasında bağlantı kurulumu.

    public void connect(String direction, Room neighbor) {
        exits.put(direction, neighbor);
    }

    // İki oda arasında kilitli bağlantı kurulumu.

    public void connectLocked(String direction, Room neighbor, String lockId)
{
        exits.put(direction, neighbor);
        lockedExits.put(direction, lockId);
    }

    // Belirtilen yöndeki çıkış döndürülür.

    public Room getExit(String direction) {
        return exits.get(direction);
    }
```

```java
    // Oda içeriği ekrana yazdılır.
    public void describe() {
        System.out.println("\n*** Şu anda bulunduğun yer: " + name + " ***");
        System.out.println(description);

        // Çıkışlar listelenir.
        System.out.print("Çıkışlar: ");
        if (exits.isEmpty()) {
            System.out.println("Hiç çıkış yok.");
        } else {
            List<String> exitDescriptions = new ArrayList<>();
            for (String direction : exits.keySet()) {
                String desc = direction;
                if (lockedExits.containsKey(direction)) {
                    desc += " (KİLİTLİ - " + lockedExits.get(direction) + ")";
                }
                exitDescriptions.add(desc);
            }
            System.out.println(String.join(", ", exitDescriptions));
        }

        // Eşyalar listelenir.
        System.out.print("Eşyalar: ");
        if (items.isEmpty()) {
            System.out.println("Hiç eşya yok.");
        } else {
            List<String> itemNames = new ArrayList<>();
            for (Item item : items) {
                itemNames.add(item.getName());
            }
            System.out.println(String.join(", ", itemNames));
        }

        // Karakterler listelenir.
        System.out.print("Karakterler: ");
        if (npcs.isEmpty()) {
            System.out.println("Hiç karakter yok.");
        } else {
            List<String> npcNames = new ArrayList<>();
            for (Npc1 npc : npcs) {
                npcNames.add(npc.getShortName());
            }
            System.out.println(String.join(", ", npcNames));
        }
        System.out.println("----------------------------------------");
    }

    // İsimle eşya aranması.
```

```java
    public Item getItem(String name) {
        for (Item item : items) {
            if (item.getName().equals(name)) {
                return item;
            }
        }
        return null;
    }

    // İsimle NPC aranması.

public Npc1 getNPC(String shortName) {
        for (Npc1 npc : npcs) {
            if (npc.getShortName().equals(shortName)) {
                return npc;
            }
        }
        return null;
    }
}
// Tuzak odası. İçeri girildiğinde oyun biter.

class TrapRoom extends Room {

    public TrapRoom(int id, String name, String description) {
        super(id, name, description);
    }

    // Odaya girildiğinde tetiklenen metot.

    public void onEnter(Player p, GameEngine ctx) {

        System.out.println("\n##########################################");
        System.out.println("Odaya girdiğin anda soğuk bir rüzgarı
hissettin.");
        System.out.println("Sen ne olduğunu anlayamadan 3 metrelik bir trol
seni gövdenden tutup havaya kaldırdı.");
        System.out.println("Korkudan donakaldın...");
        System.out.println("Seni kokladı ve sen karşı koyamadan kafanı
gövdenle birlikte ısırdı.");
        System.out.println("!!! MACERAN HAZİN BİR SON BULDU !!!");
        System.out.println("##########################################\n");

        // Oyunu bitmesi için can sıfırlanır.
        p.setHp(0);
    }
}
```

# Item.java

```java
// Oyundaki tüm eşyaların ortak özelliklerini tanımlayan soyut üst sınıf.

public abstract class Item {

    protected final String id;
    protected String name;
    protected String description;

    public Item(String id, String name, String description) {
        this.id = id;
        this.name = name;
        this.description = description;
    }

    // Dışarıdan erişim için yazılan metotlar.

    public String getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getDescription() {
        return description;
    }

    // Eşya kullanıldığında ne yapacağını tanımlayan metot.

    public abstract void onUse(Player p, GameEngine ctx);
}

// Item sınıfından türetilmiş İksir sınıfı.
class PotionItem extends Item {
    private final int healthBoost;

    public PotionItem(String id, String name, String description, int
healthBoost) {
        super(id, name, description);
        this.healthBoost = healthBoost;
    }
```

```java
    // İksir kullanıldığında oyuncunun canı artar.

    @Override
    public void onUse(Player p, GameEngine ctx) {
        p.setHp(p.getHp() + healthBoost);
        System.out.println(">>> " + this.getName() + " kullandın. Can puanın "
+ healthBoost + " arttı. Yeni canın: " + p.getHp());
        // İksir kullanıldıktan sonra envanterden kaldırılması.
        p.getInventory().remove(this);
    }
}

// Item sınıfından türetilmiş Anahtar sınıfı.

class KeyItem extends Item {
    private final String targetDoor;

    public KeyItem(String id, String name, String description, String
targetDoor) {
        super(id, name, description);
        this.targetDoor = targetDoor;
    }

    // Anahtar kullanıldığında kilitli kapı açılır.

    @Override
    public void onUse(Player p, GameEngine ctx) {
        // Hedef kapının kilidi açmaya çalışılır.
        if (ctx.unlockExit(targetDoor)) {
            System.out.println(">>> " + this.getName() + " ile kilitli bir
geçidin kilidini açtın! Artık oradan 'go' komutunu kullanarak geçebilirsin.");
        } else {
            System.out.println(">>> " + this.getName() + " hiçbir kilitli
kapıyı açmadı veya kilit zaten açıktı. Belki de doğru yerde değilsin.");
        }
    }

    public String getTargetDoor() {
        return targetDoor;
    }
}

// Item sınıfından türetilmiş Silah sınıfı.

class WeaponItem extends Item {
    private final int attackBoost;
    private boolean isEquipped; // Silahın kuşanılıp kuşanılmadığını kontrol
etmek için.
```

```java
    public WeaponItem(String id, String name, String description, int
attackBoost) {
        super(id, name, description);
        this.attackBoost = attackBoost;
        this.isEquipped = false;
    }

    // Silah kullanıldığında(kuşanılır) Saldırı gücünü artırır.

    @Override
    public void onUse(Player p, GameEngine ctx) {
        if (isEquipped) {
            // Silahı çıkarma.
            p.setAttackPower(p.getAttackPower() - attackBoost);
            isEquipped = false;
            System.out.println(">>> " + this.getName() + "'ı kınına koydun.
Mevcut saldırı gücün: " + p.getAttackPower());
        } else {
            // Silahı kuşanma.
            p.setAttackPower(p.getAttackPower() + attackBoost);
            isEquipped = true;
            System.out.println(">>> " + this.getName() + "'ı kınından
çıkardın. Saldırı gücün " + attackBoost + " arttı. Mevcut saldırı gücün: " +
p.getAttackPower());
        }
    }

    public boolean isEquipped() {
        return isEquipped;
    }
}
```

# Player.java

```java
import java.util.ArrayList;
import java.util.List;

// Oyuncunun durumunu (HP, envanter, mevcut oda, saldırı gücü) ve
etkileşimlerini yönetir.

public class Player {
    private Room currentRoom;
    private final List<Item> inventory;
    private int hp;
    private int attackPower;

    public Player(Room startRoom, int initialHp, int initialAttack) {
        this.currentRoom = startRoom;
```

```java
        this.inventory = new ArrayList<>();
        this.hp = initialHp;
        this.attackPower = initialAttack;
    }
    // Dışarıdan erişim için yazılan metotlar.
    public Room getCurrentRoom() {
        return currentRoom;
    }

    public List<Item> getInventory() {
        return inventory;
    }

    public int getHp() {
        return hp;
    }

    public void setHp(int hp) {
        this.hp = hp;
    }

    public int getAttackPower() {
        return attackPower;
    }

    public void setAttackPower(int attackPower) {
        this.attackPower = attackPower;
    }

    // Oyuncunun belirtilen yöne hareket etmesi sağlanır.

    public Room move(String direction) {
        Room nextRoom = currentRoom.getExit(direction);
        if (nextRoom == null) {
            System.out.println(">>> Bu yönde çıkış yok.");
            return null;
        }

        currentRoom = nextRoom;
        return currentRoom;
    }

    // Odadaki eşyanın envantere alınabilmesi.

    public boolean take(String itemName) {
        Item item = currentRoom.getItem(itemName);
        if (item == null) {
            System.out.println(">>> Odada böyle bir eşya (" + itemName + ")
bulunmuyor.");
```

```
            return false;
        }

        currentRoom.getItems().remove(item);
        inventory.add(item);
        System.out.println(">>> " + item.getName() + " adlı eşyayı aldın.");
        return true;
    }

    // Envanterdeki eşyanın bulunması.

    public Item findItemInInventory(String itemName) {
        for (Item item : inventory) {
            if (item.getName().equals(itemName)) {
                return item;
            }
        }
        return null;
    }

    // Envanterdeki eşyanın kullanılması.

    public boolean use(String itemName, GameEngine ctx) {
        Item item = findItemInInventory(itemName);
        if (item == null) {
            System.out.println(">>> Bu eşya (" + itemName + ") sende yok.");
            return false;
        }

        item.onUse(this, ctx);
        return true;
    }

    // Odadaki NPC ile konuşma.

    public boolean talkToNPC(String npcName, GameEngine ctx) {
        Npc1 npc = currentRoom.getNPC(npcName);
        if (npc == null) {
            System.out.println(">>> Odada böyle bir karakter (" + npcName + ")
bulunmuyor.");
            return false;
        }

        npc.talk(this, ctx);
        return true;
    }

    // Oyuncu envanterinin ve mevcut durumunun listelenmesi.
```

```java
    public void inventoryText() {
        System.out.println("\n--- ENVANTER ---");
        System.out.println("Can Puanın (HP): " + hp);
        System.out.println("Saldırı Gücün (ATK): " + attackPower);

        if (inventory.isEmpty()) {
            System.out.println("Envanterin boş.");
        } else {
            System.out.println("Taşıdığın Eşyalar:");
            for (Item item : inventory) {
                System.out.println(" - " + item.getName() );
            }
        }
        System.out.println("-----------------\n");
    }
}
```

# Npc1.java

```java
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

// Oyundaki NPC'lerin temel özelliklerini tanımlar.

public abstract class Npc1{
    protected String name;
    protected String description;

    public Npc1(String name, String description) {
        this.name = name;
        this.description = description;
    }
    // Dışarıdan erişim için yazılan metotlar.
    public String getName() {
        return name;
    }

        public String getShortName() {
        return name.toLowerCase().replace(" ", "_").replaceAll("[ıİ]",
"i").replaceAll("[üÜ]", "u").replaceAll("[öÖ]", "o").replaceAll("[çÇ]",
"c").replaceAll("[ğĞ]", "g").replaceAll("[şŞ]", "s");
    }
    public String getDescription() {
        return description;
    }
```

```java
    // Oyuncu bu karakterle konuşmaya çalıştığında ne olacağını belirler.
    public abstract void talk(Player p, GameEngine ctx);
}

// NPC sınıfından türetilmiş Dost Karakter sınıfı.

class FriendlyNPC extends Npc1 {
    private final ConversationNode dialogueRoot;

    public FriendlyNPC(String name, String description, ConversationNode
dialogueRoot) {
        super(name, description);
        this.dialogueRoot = dialogueRoot;
    }

    // Dost karakterle konuşulduğunda diyalog ağacında ilerlenir.

    @Override
    public void talk(Player p, GameEngine ctx) {
        System.out.println(">>> " + this.name + " ile konuşuyorsun.");

        ConversationNode currentNode = dialogueRoot;
        Scanner scanner = new Scanner(System.in);

        while (currentNode != null && !currentNode.getChoices().isEmpty()) {
            currentNode.display();

            // Oyuncunun seçenek numarası girerek diyaloga yün vermesi.
            System.out.print("Seçimini yap (1-" +
currentNode.getChoices().size() + ") veya (cikis): ");
            String input = scanner.nextLine().trim().toLowerCase();
            scanner.close();
            if (input.equals("cikis")) {
                System.out.println(">>> Konuşmayı sonlandırdın.");
                return;
            }

            try {
                int choiceIndex = Integer.parseInt(input) - 1;
                List<String> choices = currentNode.getChoices();

                if (choiceIndex >= 0 && choiceIndex < choices.size()) {
                    String selectedChoiceText = choices.get(choiceIndex);
                    // Seçilen seçeneğe karşılık gelen sonraki düğümün
alınması.
                    currentNode = currentNode.getNextNode(selectedChoiceText);

                    if (currentNode == null) {
```

```
                        System.out.println(">>> " + this.name + ": Anladım. Bu
konuda söyleyeceklerim bu kadardı.");
                        break;
                    }

                } else {
                    System.out.println(">>> Geçersiz seçim. Lütfen geçerli bir
numara gir.");
                }
            } catch (NumberFormatException e) {
                System.out.println(">>> Geçersiz komut. Lütfen bir numara veya
'cikis' yazın.");
            }
        }

        if (currentNode != null) {
            currentNode.display();
        }
    }

    // Dost NPC oluşturulması. İki adımlı bir diyalog ağacı kurulur.

    public static FriendlyNPC createWiseMan() {

        ConversationNode nodeGoodbye = new ConversationNode(
            "Görüşmek üzere gezgin. Umarım aradığını bulursun.",
            null
        );

        ConversationNode nodeKeyInfo = new ConversationNode(
            "Anahtar, Salon'un hemen batısındaki Mutfak'ta saklıdır. Dikkatli
ol, etrafta tuzaklar olabilir.",
            Map.of("Teşekkürler, gidiyorum.", nodeGoodbye)
        );

        ConversationNode nodeGateInfo = new ConversationNode(
            "Karanlık Kapı, Koridor'un batı ucundaki kilitli geçittir. Oraya
ulaşmak için kilitli anahtarı bulmalısın.",
            Map.of("Peki, bana şans dile.", nodeGoodbye)
        );

        Map<String, ConversationNode> rootChoices = new HashMap<>();
        rootChoices.put("Anahtar nerede?", nodeKeyInfo);
        rootChoices.put("Karanlık Kapı da ne?", nodeGateInfo);
        rootChoices.put("Hoşça kal.", nodeGoodbye);

        ConversationNode nodeRoot = new ConversationNode(
            "Selam gezgin. Bu diyarları görmeyeli uzun zaman oldu. Karanlık
Kapı'dan geçmek istiyorsan, kırmızı anahtarı bulmalısın.",
```

```
            rootChoices
        );

        return new FriendlyNPC("Bilge Adam", "Odanın köşesinde oturmuş yaşlı
ve bilge görünümlü bir adam.", nodeRoot);

    }

}
// NPC sınıfından türetilmiş Düşman Karakter sınıfı.


 class EnemyNPC extends Npc1 {
    private final int attackPower;

    public EnemyNPC(String name, String description, int attackPower) {
        super(name, description);
        this.attackPower = attackPower;
    }

    // Düşman karakterle konuşulduğunda tehdit eder ve hemen ardından
saldırır.

    @Override
    public void talk(Player p, GameEngine ctx) {
        System.out.println(">>> " + this.name + " sana hırlıyor ve kılıcını
çekiyor!");
        System.out.println(">>> \"Git buradan! Yoksa...\" der ve anında
saldırır!");

        attack(p);
    }

    // Düşmanın oyuncuya saldırma metotu.

    public void attack(Player p) {
        int damage = this.attackPower;

        p.setHp(p.getHp() - damage);

        System.out.println("<<< Düşman (" + this.name + ") sana " + damage + "
hasar verdi.");
        System.out.println("<<< Kalan Can Puanın: " + p.getHp());

        if (p.getHp() <= 0) {
            System.out.println("!!! Düşman seni mağlup etti. Canın kalmadı.");
        }
    }
```

```
    // Düşman NPC oluşturulması.

    public static EnemyNPC createGuard() {
        return new EnemyNPC("muhafız", "Ağır zırh giymiş, kılıçlı, gözleri
tehditkar bir muhafız.", 10);
    }
}
```

# ConversationNode.java

```
import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

/**
 * NPC diyalog ağacındaki bir düğümü temsil eder.
 * Bir metin ve sonraki olası diyalog düğümlerine (seçenekler) geçişi tutar.
 */
class ConversationNode {
    private final String text;
    private final Map<String, ConversationNode> nextNodes;

    public ConversationNode(String text, Map<String, ConversationNode>
nextNodes) {
        this.text = text;
        this.nextNodes = nextNodes != null ? nextNodes :
Collections.emptyMap();
    }

    public String getText() {
        return text;
    }

    // Oyuncunun görebileceği seçenek metinlerinin döndürülmesi.
    public List<String> getChoices() {
        return nextNodes.keySet().stream().collect(Collectors.toList());
    }

    // Belirtilen seçenek metnine karşılık gelen bir sonraki düğümün
döndürülmesi.

    public ConversationNode getNextNode(String choiceText) {
        return nextNodes.get(choiceText);
    }

    // Diyalog düğümünün ekrana yazdırılması.
```

```java
    public void display() {
        System.out.println("\n--- Diyalog ---");
        System.out.println(text);

        List<String> choices = getChoices();
        if (!choices.isEmpty()) {
            System.out.println("\nSeçenekler:");
            // Seçenekleri numaralandırarak gösteririz
            for (int i = 0; i < choices.size(); i++) {
                System.out.println("  [" + (i + 1) + "] " + choices.get(i));
            }
        } else {
            System.out.println("(Diyalog sona erdi.)");
        }
        System.out.println("---------------\n");
    }
}
```

# GameEngine.java

```java
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;


// Oyunun ana akışının, komut işlemeyi ve dünya yapısının (Map) yönetilmesi.
public class GameEngine {
    private Player player;
    private final Map<Integer, Room> map;
    private boolean isRunning;
    private final Map<String, Boolean> lockStates;

    public GameEngine() {
        this.map = new HashMap<>();
        this.lockStates = new HashMap<>();
        this.isRunning = true;
    }

    // Oyunun başlatması. Odaların, eşyaların, NPC'lerin oluşturulması ve
bağlantıların kurulması.

    public void initGame() {
        System.out.println("Mini Macera Oyunu Başlatılıyor...");

        // Odaları oluşturulması
        Room salon = new Room(1, "Salon", "Büyük bir salon. Köşede yaşlı bir
adam oturuyor. Duvarlarda eski tablolar asılı.");
```

```
        Room koridor = new Room(2, "Koridor", "Dar ve uzun bir koridor. Batı
ucu karanlık bir kapıya açılıyor.");
        Room mutfak = new Room(3, "Mutfak", "Kullanılmayan, tozlu bir mutfak.
Tezgâhın üzerinde parlayan bir şey var.");
        Room kütüphane = new Room(4, "Kütüphane", "Tavanlara kadar uzanan
kitap raflarıyla dolu sessiz bir kütüphane.");
        Room mahzen = new Room(5, "Mahzen", "Rutubetli, soğuk bir mahzen.
Burada bir muhafız nöbet tutuyor.");
        Room gizliOda = new Room(6, "Gizli Oda", "Başarıyla ulaştın! Burası
maceranın sonu. İçeride büyük bir hazine var.");
        TrapRoom tuzakOdasi = new TrapRoom(7, "Kanlık Oda", "");

        // Haritaya odaları ID'leri ile eklemenmesi
        map.put(salon.getId(), salon);
        map.put(koridor.getId(), koridor);
        map.put(mutfak.getId(), mutfak);
        map.put(kütüphane.getId(), kütüphane);
        map.put(mahzen.getId(), mahzen);
        map.put(gizliOda.getId(), gizliOda);
        map.put(tuzakOdasi.getId(), tuzakOdasi);

        // Odalar arası bağlantıların kurulması
        salon.connect("kuzey", koridor);
        salon.connect("bati", mutfak);

        koridor.connect("guney", salon);
        koridor.connect("dogu", kütüphane);
        koridor.connectLocked("bati", mahzen, "kirmizi_anahtar_kilidi");
        lockStates.put("kirmizi_anahtar_kilidi", true);

        mutfak.connect("dogu", salon);
        mutfak.connect("kuzey", tuzakOdasi);

        kütüphane.connect("bati", koridor);
        kütüphane.connect("kuzey", gizliOda);

        mahzen.connect("dogu", koridor);

        // Itemlerin oluşturulması
        Item key = new KeyItem("item_key_1", "kirmizi_anahtar", "Üzerinde
tuhaf semboller olan paslı bir anahtar.", "kirmizi_anahtar_kilidi");
        Item potion = new PotionItem("item_potion_1", "saglik_iksiri", "Canını
15 puan artırır.", 15);
        Item sword = new WeaponItem("item_weapon_1", "eski_kilic", "Paslı ama
güçlü bir kılıç. Saldırı gücünü 10 artırır.", 10);

        // Odalara itemlerin eklenmesi
        mutfak.getItems().add(key);
        kütüphane.getItems().add(potion);
```

```java
        mahzen.getItems().add(sword);

        // NPC'lerin oluşturulması
        Npc1 wiseMan = FriendlyNPC.createWiseMan();
        Npc1 guard = EnemyNPC.createGuard();

        // NPC'lerin odalara yerleştirilmesi
        salon.getNpcs().add(wiseMan);
        mahzen.getNpcs().add(guard);

        // Oyuncunun oluşturulması
        player = new Player(salon, 100, 1);
        System.out.println("Oyun hazır. Başarılar dileriz!\n");
        player.getCurrentRoom().describe();
    }

    // Belirtilen kilit ID'sine sahip çıkışın kilidinin açılması.

    public boolean unlockExit(String lockId) {
        if (lockStates.containsKey(lockId) && lockStates.get(lockId)) {
            lockStates.put(lockId, false);
            return true;
        }
        return false;
    }

    // Ana oyun döngüsünün çalıştırılması.
    public void startGameLoop() {
        Scanner scanner = new Scanner(System.in);
        printHelp();

        while (isRunning) {
            System.out.print("\n> ");
            try { // Sistemsel istisnalar
                String input = scanner.nextLine().trim().toLowerCase();
                if (input.isEmpty()) continue;

                processCommand(input);

                // Kazanma koşulu kontrolü
                if (player.getCurrentRoom().getName().equals("Gizli Oda")) {
                    System.out.println("\n*** TEBRİKLER! ***");
                    System.out.println("Gizli Odaya ulaştın ve büyük hazineyi
buldun!");
                    System.out.println("Maceran sona erdi.");
                    isRunning = false;
                }

                // Oyuncu Can kontrolü
```

```
                if (player.getHp() <= 0) {
                     System.out.println("\n*** OYUN BİTTİ ***");
                     // Özel tuzak odası mesajı
                     if (!player.getCurrentRoom().getName().equals("Tuzak
Odası")) {
                         System.out.println("Can puanın sıfırlandı. Maalesef
maceran burada sona erdi.");
                     }
                     isRunning = false;
                }

            } catch (Exception e) {
                System.out.println("!!! HATA: Beklenmedik bir sorun oluştu.
Detay: " + e.getMessage());
            }
        }
        // Oyun sonu özeti
        System.out.println("\n=== OYUN ÖZETİ ===");
        System.out.println("Can Puanın: " + player.getHp());
        System.out.println("Saldırı Gücün: " + player.getAttackPower());
        System.out.println("Son konumun: " +
player.getCurrentRoom().getName());
        scanner.close();
    }

    // Kullanıcıdan alınan komutun işlenmesi ve ilgili sınıfların
tetiklenmesi.

    private void processCommand(String commandText) {
        String[] parts = commandText.split(" ");
        String command = parts[0];
        // Sadece ikinci kelimenin yakalanması
        String argument = parts.length > 1 ? parts[1] : "";

        Room currentRoom = player.getCurrentRoom();

        switch (command) {
            case "look":
                currentRoom.describe();
                break;

            case "go":
                if (argument.isEmpty()) {
                    System.out.println(">>> Gitmek istediğin yönü
belirtmelisin. Örnek: go kuzey");
                    return;
                }

                // Kilitli mi diye kontrol edilmesi
```

```
                if (currentRoom.getLockedExits().containsKey(argument) &&
lockStates.getOrDefault(currentRoom.getLockedExits().get(argument), false)) {
                    System.out.println(">>> Bu yöndeki (" + argument + ")
geçit kilitli.");
                    return;
                }

                Room nextRoom = currentRoom.getExit(argument);
                if (nextRoom != null) {
                    player.move(argument);

                    // Odaya girildikten sonra kontrol
                    Room enteredRoom = player.getCurrentRoom();

                    if (enteredRoom instanceof TrapRoom) {
                        // Odaya girme olayının tetiklenmesi ve oyunun
bitirilmesi
                        ((TrapRoom) enteredRoom).onEnter(player, this);
                    }

                    // Oda açıklamasının yazdırılması
                    enteredRoom.describe();
                } else {
                    System.out.println(">>> Bu yönde çıkış yok.");
                }
                break;

            case "take":
                if (argument.isEmpty()) {
                    System.out.println(">>> Almak istediğin eşyayı
belirtmelisin. Örnek: take anahtar");
                    return;
                }
                player.take(argument);
                break;

            case "use":
                if (argument.isEmpty()) {
                    System.out.println(">>> Kullanmak istediğin eşyayı
belirtmelisin. Örnek: use iksir");
                    return;
                }
                // KeyItem kullanımında kilit açma ve geçiş GameEngine
tarafından sağlanır.
                player.use(argument, this);
                break;

            case "talk":
                if (argument.isEmpty()) {
```

```
                    System.out.println(">>> Konuşmak istediğin karakteri
belirtmelisin. Örnek: talk muhafiz");
                    return;
                }
                player.talkToNPC(argument, this);
                break;

            case "say":
                String sayText =
commandText.substring(command.length()).trim();

                if (sayText.isEmpty()) {
                    System.out.println(">>> Ne söylemek istediğini
belirtmelisin. Örnek: say merhaba dünya");
                    return;
                }

                System.out.println(">>> Şunu söylüyorsun: \"" + sayText +
"\"");

                break;

            case "inv":
                player.inventoryText();
                break;

            case "help":
                printHelp();
                break;

            case "quit":
                isRunning = false;
                System.out.println(">>> Oyundan çıkılıyor...");
                break;

            default:
                System.out.println(">>> Bilinmeyen komut. 'help' yazarak
komutları görebilirsin.");
                break;
        }
    }

    // Kullanılabilecek komutların ekrana yazdırılması.

    private void printHelp() {
        System.out.println("\nKullanılabilir Komutlar:");
        System.out.println("----------------------------------------");
        System.out.println("  look            : Odanın detaylarını (eşya, NPC,
çıkışlar) tekrar listeler.");
```

```
        System.out.println("  go (yön)        : Belirtilen yöne gider (kuzey,
guney, bati, dogu).");
        System.out.println("  take (eşya)     : Odadaki bir eşyayı envantere
alır.");
        System.out.println("  use (eşya)      : Envanterdeki bir eşyayı
kullanır.");
        System.out.println("  talk (karakter) : Odadaki bir NPC ile
konuşur.");
        System.out.println("  say (metin)     : Ortama bir şey söylersin.");
        System.out.println("  inv             : Envanterini ve can puanını
gösterir.");
        System.out.println("  help            : Bu yardım menüsünü
gösterir.");
        System.out.println("  quit            : Oyunu sonlandırır.");
        System.out.println("----------------------------------------");
    }
}
```

# App.java

```
// Programın başlangıç noktası. GameEngine'nin başlatılması ve ana döngünün
çağırılması.

public class App {
    public static void main(String[] args) {
        // Oyun motoru nesnesinin oluşturulması
        GameEngine game = new GameEngine();

        // Oyunun başlatılması ve dünya haritasının kurulması
        game.initGame();

        // Ana oyun döngüsünün çalıştırılması
        game.startGameLoop();
    }
}
```

# 5. Çıktılar (Oyun Oturumu Örnekleri)

## Senaryo 1: Gizli Odaya Ulaşmak

```
Mini Macera Oyunu Başlatılıyor...
Oyun hazır. Başarılar dileriz!


*** Şu anda bulunduğun yer: Salon ***
Büyük bir salon. Köşede yaşlı bir adam oturuyor. Duvarlarda eski tablolar asılı.
Çıkışlar: bati, kuzey
Eşyalar: Hiç eşya yok.
Karakterler: bilge_adam
----------------------------------------

Kullanılabilir Komutlar:
----------------------------------------
  look            : Odanın detaylarını (eşya, NPC, çıkışlar) tekrar listeler.
  go (yön)        : Belirtilen yöne gider (kuzey, guney, bati, dogu).
  take (eşya)     : Odadaki bir eşyayı envantere alır.
  use (eşya)      : Envanterdeki bir eşyayı kullanır.
  talk (karakter) : Odadaki bir NPC ile konuşur.
  say (metin)     : Ortama bir şey söylersin.
  inv             : Envanterini ve can puanını gösterir.
  help            : Bu yardım menüsünü gösterir.
  quit            : Oyunu sonlandırır.
----------------------------------------

> look

*** Şu anda bulunduğun yer: Salon ***
Büyük bir salon. Köşede yaşlı bir adam oturuyor. Duvarlarda eski tablolar asılı.
Çıkışlar: bati, kuzey
Eşyalar: Hiç eşya yok.
Karakterler: bilge_adam
----------------------------------------

> talk bilge_adam
>>> Bilge Adam ile konuşuyorsun.

--- Diyalog ---
Selam gezgin. Bu diyarları görmeyeli uzun zaman oldu. Karanlık Kapı'dan geçmek istiyorsan, kırmızı anahtarı bulmalısın.
```

```
Seçenekler:
  [1] Karanlık Kapı da ne?
  [2] Anahtar nerede?
  [3] Hoşça kal.
--------------

Seçimini yap (1-3) veya (cikis): 1

-- Diyalog ---
Karanlık Kapı, Koridor'un batı ucundaki kilitli geçittir. Oraya ulaşmak için kilitli anahtarı bulmalısın.

Seçenekler:
  [1] Peki, bana şans dile.
--------------

Seçimini yap (1-1) veya (cikis): 1

-- Diyalog ---
Görüşmek üzere gezgin. Umarım aradığını bulursun.
(Diyalog sona erdi.)
--------------


> go bati

*** Şu anda bulunduğun yer: Mutfak ***
Kullanılmayan, tozlu bir mutfak. Tezgâhın üzerinde parlayan bir şey var.
Çıkışlar: dogu, kuzey
Eşyalar: kirmizi_anahtar
Karakterler: Hiç karakter yok.
----------------------------------------

> take kirmizi_anahtar
>>> kirmizi_anahtar adlı eşyayı aldın.

> go dogu
*** Şu anda bulunduğun yer: Mutfak ***
Kullanılmayan, tozlu bir mutfak. Tezgâhın üzerinde parlayan bir şey var.
Çıkışlar: dogu, kuzey
Eşyalar: kirmizi_anahtar
Karakterler: Hiç karakter yok.
----------------------------------------

> take kirmizi_anahtar
>>> kirmizi_anahtar adlı eşyayı aldın.

> go dogu

*** Şu anda bulunduğun yer: Salon ***
Büyük bir salon. Köşede yaşlı bir adam oturuyor. Duvarlarda eski tablolar asılı.
Çıkışlar: bati, kuzey
Eşyalar: Hiç eşya yok.
Karakterler: bilge_adam
----------------------------------------

> go kuzey

*** Şu anda bulunduğun yer: Koridor ***
Dar ve uzun bir koridor. Batı ucu karanlık bir kapıya açılıyor.
Çıkışlar: guney, dogu, bati (KİLİTLİ - kirmizi_anahtar_kilidi)
Eşyalar: Hiç eşya yok.
Karakterler: Hiç karakter yok.
----------------------------------------

> go dogu

*** Şu anda bulunduğun yer: Kütüphane ***
Tavanlara kadar uzanan kitap raflarıyla dolu sessiz bir kütüphane.
Çıkışlar: bati, kuzey
Eşyalar: saglik_iksiri
Karakterler: Hiç karakter yok.
----------------------------------------
```

```
> go kuzey

*** Şu anda bulunduğun yer: Gizli Oda ***
Başarıyla ulaştın! Burası maceranın sonu. İçeride büyük bir hazine var.
Çıkışlar: Hiç çıkış yok.
Eşyalar: Hiç eşya yok.
Karakterler: Hiç karakter yok.
----------------------------------------

*** TEBRİKLER! ***
Gizli Odaya ulaştın ve büyük hazineyi buldun!
Maceran sona erdi.

=== OYUN ÖZETİ ===
Can Puanın: 100
Saldırı Gücün: 1
Son konumun: Gizli Oda
```

# Senaryo 2: NPC ile Savaş

```
Mini Macera Oyunu Başlatılıyor...
Oyun hazır. Başarılar dileriz!


*** Şu anda bulunduğun yer: Salon ***
Büyük bir salon. Köşede yaşlı bir adam oturuyor. Duvarlarda eski tablolar asılı.
Çıkışlar: bati, kuzey
Eşyalar: Hiç eşya yok.
Karakterler: bilge_adam
----------------------------------------

Kullanılabilir Komutlar:
----------------------------------------
  look            : Odanın detaylarını (eşya, NPC, çıkışlar) tekrar listeler.
  go (yön)        : Belirtilen yöne gider (kuzey, guney, bati, dogu).
  take (eşya)     : Odadaki bir eşyayı envantere alır.
  use (eşya)      : Envanterdeki bir eşyayı kullanır.
  talk (karakter) : Odadaki bir NPC ile konuşur.
  say (metin)     : Ortama bir şey söylersin.
  inv             : Envanterini ve can puanını gösterir.
  help            : Bu yardım menüsünü gösterir.
  quit            : Oyunu sonlandırır.
----------------------------------------

> go bati

*** Şu anda bulunduğun yer: Mutfak ***
Kullanılmayan, tozlu bir mutfak. Tezgâhın üzerinde parlayan bir şey var.
Çıkışlar: dogu, kuzey
Eşyalar: kirmizi_anahtar
Karakterler: Hiç karakter yok.
----------------------------------------

> take kirmizi_anahtar
>>> kirmizi_anahtar adlı eşyayı aldın.

> go dogu

*** Şu anda bulunduğun yer: Salon ***
Büyük bir salon. Köşede yaşlı bir adam oturuyor. Duvarlarda eski tablolar asılı.
Çıkışlar: bati, kuzey
Eşyalar: Hiç eşya yok.
Karakterler: bilge_adam
----------------------------------------

> go kuzey

*** Şu anda bulunduğun yer: Koridor ***
Dar ve uzun bir koridor. Batı ucu karanlık bir kapıya açılıyor.
Çıkışlar: guney, dogu, bati (KİLİTLİ - kirmizi_anahtar_kilidi)
Eşyalar: Hiç eşya yok.
Karakterler: Hiç karakter yok.
----------------------------------------

> use kirmizi_anahtar
>>> kirmizi_anahtar ile kilitli bir geçidin kilidini açtın! Artık oradan 'go' komutunu kullanarak geçebilirsin.

> go bati

*** Şu anda bulunduğun yer: Mahzen ***
Rutubetli, soğuk bir mahzen. Burada bir muhafız nöbet tutuyor.
Çıkışlar: dogu
Eşyalar: eski_kilic
Karakterler: muhafiz
----------------------------------------

> take eski_kilic
>>> eski_kilic adlı eşyayı aldın.

> use eski_kilic
>>> eski_kilic'i kınından çıkardın. Saldırı gücün 10 arttı. Mevcut saldırı gücün: 11

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
```

```
<<< Kalan Can Puanın: 90

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 80

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 70

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 60

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 50

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 40

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 30

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 20

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 10

> talk muhafiz
>>> muhafız sana hırlıyor ve kılıcını çekiyor!
>>> "Git buradan! Yoksa..." der ve anında saldırır!
<<< Düşman (muhafız) sana 10 hasar verdi.
<<< Kalan Can Puanın: 0
!!! Düşman seni mağlup etti. Canın kalmadı.

*** OYUN BİTTİ ***
Can puanın sıfırlandı. Maalesef maceran burada sona erdi.

=== OYUN ÖZETİ ===
Can Puanın: 0
Saldırı Gücün: 11
Son konumun: Mahzen
```

## Senaryo 3: Tuzak Odası (Oyun Sonu)

```
Mini Macera Oyunu Başlatılıyor...
Oyun hazır. Başarılar dileriz!


*** Şu anda bulunduğun yer: Salon ***
Büyük bir salon. Köşede yaşlı bir adam oturuyor. Duvarlarda eski tablolar asılı.
Çıkışlar: bati, kuzey
Eşyalar: Hiç eşya yok.
Karakterler: bilge_adam
---------------------------------------

Kullanılabilir Komutlar:
---------------------------------------
  look            : Odanın detaylarını (eşya, NPC, çıkışlar) tekrar listeler.
  go (yön)        : Belirtilen yöne gider (kuzey, guney, bati, dogu).
  take (eşya)     : Odadaki bir eşyayı envantere alır.
  use (eşya)      : Envanterdeki bir eşyayı kullanır.
  talk (karakter) : Odadaki bir NPC ile konuşur.
  say (metin)     : Ortama bir şey söylersin.
  inv             : Envanterini ve can puanını gösterir.
  help            : Bu yardım menüsünü gösterir.
  quit            : Oyunu sonlandırır.
---------------------------------------

> go bati

*** Şu anda bulunduğun yer: Mutfak ***
Kullanılmayan, tozlu bir mutfak. Tezgâhın üzerinde parlayan bir şey var.
Çıkışlar: dogu, kuzey
Eşyalar: kirmizi_anahtar
Karakterler: Hiç karakter yok.
---------------------------------------

> go kuzey

##########################################
Odaya girdiğin anda soğuk bir rüzgarı hissettin.
Sen ne olduğunu anlayamadan 3 metrelik bir trol seni gövdenden tutup havaya kaldırdı.
Korkudan donakaldın...
Seni kokladı ve sen karşı koyamadan kafanı gövdenle birlikte ısırdı.
!!! MACERAN HAZİN BİR SON BULDU !!!
##########################################


*** Şu anda bulunduğun yer: Kanlık Oda ***

Çıkışlar: Hiç çıkış yok.
Eşyalar: Hiç eşya yok.
Karakterler: Hiç karakter yok.
---------------------------------------

*** OYUN BİTTİ ***
Can puanın sıfırlandı. Maalesef maceran burada sona erdi.

=== OYUN ÖZETİ ===
Can Puanın: 0
Saldırı Gücün: 1
Son konumun: Kanlık Oda
```

# 6. Sonuç ve Değerlendirme

Bu proje ile Java programlama dilinde nesne tabanlı tasarımın gücü ortaya konulmuştur. Özellikle:

- Oda ve Eşya sistemlerinde **manuel ID ataması** yapılarak veri tutarlılığı sağlanmıştır.
- **Tuzak Odası** eklenerek kalıtım (inheritance) yapısının oyun mekaniklerini nasıl zenginleştirdiği gösterilmiştir.
- Sonuç olarak, genişletilebilir, modüler ve hata yönetimi güçlü bir oyun altyapısı oluşturulmuştur.